Report

v. 1.0

Customer

RedStone

Smart Contract Audit

MultiFeed Adapter

8th January 2026

Report prepared by

ABDK Consulting

Contents

1 Changelog

#	Date	Author	Description
0.1	08.01.26	A. Zveryanskaya	Initial Draft
0.2	08.01.26	A. Zveryanskaya	Minor revision
1.0	08.01.26	A. Zveryanskaya	Release

2 Summary

This document presents the results of a smart contract audit performed by ABDK Consulting for RedStone. The audit was conducted by Mikhail Vladimirov and Dmitry Khovratovich from 26th November 2025 to 7th January 2025. The goal of the audit was to assess the security, correctness, and code quality of the `MultiFeedAdapterWithoutRoundsWithReference` contract, which implements a dual-adapter oracle system designed to improve stability while introducing new, faster oracle nodes and relayers by combining a main adapter with a reference adapter for sanity checks.

Our conclusion is that the System demonstrates solid engineering practices and represents a mature implementation of a dual-adapter oracle mechanism. The codebase exhibits good overall quality with clear logic flow and appropriate error handling. The contract is written in Solidity and implements the `IMultiFeedAdapter` interface, utilizing gas-limited external calls and deviation-based switching logic between adapters.

During our review, we identified three moderate severity issues. The moderate findings include a batch operation revert scenario where failure of both adapters for a single data feed causes the entire batch to fail, suboptimal execution of deviation checking logic when reference data is already fresher, and a potential phantom overflow in the deviation calculation function.

Moderate issues are provided for recommendation purposes. The contract demonstrates thoughtful design in managing potential gas consumption attacks through configurable limits and provides clear fallback mechanisms. Overall, the System is well-structured and suitable for production use after addressing the identified moderate issues.

It is important to note that a security audit is not a guarantee of absolute security but rather a snapshot of the system's security posture at a specific point in time. While we strive to uncover all potential issues, the evolving nature of blockchain technology and DeFi means new vulnerabilities can emerge.

3 System overview

This section provides a high-level overview of the dual-adapter oracle mechanism implemented in the audited codebase.

We were asked to review the code provided as a separate file and the commit with fixes:

MultiFeedAdapterWithoutRoundsWith Reference.sol

The System is designed to improve the stability and reliability of oracle data feeds by introducing a redundancy layer that combines a primary data source with a reference data source for validation and fallback purposes. The core functionality enables consumers to query price or data feed values while automatically handling scenarios where the primary source may fail, become stale, or show unexpected deviations from expected values.

The codebase is implemented in Solidity version 0.8.17, leveraging the language's built-in overflow and underflow protection mechanisms. The System consists of a single abstract contract that serves as a foundation for specific adapter implementations, allowing for flexible deployment across different oracle infrastructures while maintaining consistent validation logic.

The System architecture centers on coordinating two independent data adapters: a main adapter intended to provide faster, more frequently updated data, and a reference adapter that serves as a proven, stable fallback. When a data feed is queried, the System first attempts to retrieve values from both adapters independently. If both sources successfully return data, the System performs a deviation comparison to detect any significant discrepancies between the two values. The switching logic evaluates whether the reference adapter's data is sufficiently fresh based on configurable time thresholds and whether the deviation exceeds acceptable limits defined in basis points. The System includes protective measures against potential gas consumption attacks by limiting the computational resources allocated to each adapter query. When both adapters provide healthy data without excessive deviation, the System defaults to returning the most recently updated value, ensuring consumers receive the freshest available information. The contract also provides batch query capabilities for monitoring tools, allowing multiple data feeds to be checked simultaneously. External integrations are abstracted through a standard interface that both the main and reference adapters must implement, enabling the System to work with various oracle node infrastructures without requiring modifications to the core validation logic.

4 Methodology

The methodology is not a strict formal procedure, but rather a selection of methods and tactics combined differently and tuned for each particular project, depending on the project structure and technologies used, as well as on client expectations from the audit.

- **General Code Assessment**. The code is reviewed for clarity, consistency, style, and for whether it follows best code practices applicable to the particular programming language used. We check indentation, naming convention, commented code blocks, code duplication, confusing names, confusing, irrelevant, or missing comments etc. At this phase we also understand overall code structure.
- **Entity Usage Analysis**. Usages of various entities defined in the code are analysed. This includes both: internal usages from other parts of the code as well as potential external usages. We check that entities are defined in proper places as well as their visibility scopes and access levels are relevant. At this phase, we understand overall system architecture and how different parts of the code are related to each other.
- **Access Control Analysis**. For those entities, that could be accessed externally, access control measures are analysed. We check that access control is relevant and done properly. At this phase, we understand user roles and permissions, as well as what assets the system ought to protect.
- **Code Logic Analysis**. The code logic of particular functions is analysed for correctness and efficiency. We check whether the code actually does what it is supposed to do, whether the algorithms are optimal and correct, and whether proper data types are used. We also make sure that external libraries used in the code are up to date and relevant to the tasks they solve in the code. At this phase we also understand data structures used and the purposes they are used for.

We classify issues by the following severity levels:

- **Critical issue** directly affects the smart contract functionality and may cause a significant loss.
- **Major issue** is either a solid performance problem or a sign of misuse: a slight code modification or environment change may lead to loss of funds or data. Sometimes it is an abuse of unclear code behaviour which should be double checked.
- **Moderate issue** is not an immediate problem, but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain business flows.
- **Recommendations** cover code style, best practices and general improvements.

5 Our findings

We've provided the client with a few recommendations.

Moderate	Info	Fixed
	2	1

6 Moderate Issues

CVF-1 INFO

- **Category** Suboptimal
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description In case for some of the data feeds both adapters fail, the whole batch will be reverted and no data will be returned even for healthy data feeds.

Recommendation Refactor the code to return data for healthy data feeds even if some data feeds in a batch failed.

Client Comment *It would require changing the interface, because we would need to pass information what feeds are missing. This function is designed to be used by monitoring tools, not by other contracts*

```
111 (detailsForFeeds[i].dataTimestamp, detailsForFeeds[i].blockTimestamp
    ↪ , detailsForFeeds[i].value) = getLastUpdateDetails(dataFeedIds
    ↪ [i]);
```

CVF-2 FIXED

- **Category** Suboptimal
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description All this logic shouldn't be executed in case refDataTimestamp > mainDataTimestamp.

```
66 (uint256 maxAllowedDeviationBps, uint256 maxDataAgeInSeconds) =
    ↪ getReferenceSwitchCriteria(dataFeedId);
uint256 deviation = calculateDeviationBpsSafe(mainValue, refValue);
bool isRefDataFresh = (block.timestamp - refBlockTimestamp) <=
    ↪ maxDataAgeInSeconds;
```

```
70 // Due to the freshness check we should push reference data more
    ↪ often than `maxDataAgeInSeconds`
if (deviation > maxAllowedDeviationBps && isRefDataFresh) {
  return (refDataTimestamp, refBlockTimestamp, refValue);
}
```

CVF-3 INFO

- **Category** Overflow/Underflow
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description Phantom overflow is possible here, i.e. a situation when the final calculation result would fit into the destination type, while certain intermediary calculation overflows.

Client Comment *The value "type(uint256).max / 10000" is approximately 10^73, so we have a large reserve even if we significantly increase the value in the decimals() function*

```
91 return ((maxVal - minVal) * 10_000) / maxVal;
```

7 Recommendations

CVF-4 INFO

- **Category** Procedural
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description Consider specifying as "^0.8.0" unless there is something special regarding this particular version.

Client Comment *We use this version for all our contracts due to the issues with 0.8.13*

```
3 pragma solidity ^0.8.17;
```

CVF-5 INFO

- **Category** Bad naming
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description The contract name is tool long and contains too much details.

Recommendation Use shorter name.

```
15 abstract contract MultiFeedAdapterWithoutRoundsWithReference is
   ↪ IMultiFeedAdapter {
```

CVF-6 INFO

- **Category** Bad naming
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description The constant name is too long and contain too much details.

Recommendation Use shorter name.

```
16 uint256 internal constant DEFAULT_GET_LAST_UPDATE_DETAILS_GAS_LIMIT
   ↪ = 2_000_000;
```

CVF-7 FIXED

- **Category** Suboptimal
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description These errors could be made more useful by adding certain parameters into them.

Client Comment *We will add the dataFeedId parameter, because other values (ok, dataTimestamp, blockTimestamp, value) for both adapters are already known: (false, 0, 0, 0). Regarding UnsupportedFunctionCall, we don't need any parameters, because we intentionally do not want to use those functions*

```
18 error UnsupportedFunctionCall();
   error BothAdaptersFailed();
```

CVF-8 FIXED

- **Category** Unclear behavior
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description This 7/10 quotient looks arbitrary.

Recommendation Make is a named constant and explain in a comment how this value was chosen.

```
43 return _max(gasleft() * 7 / 10,
     ↪ DEFAULT_GET_LAST_UPDATE_DETAILS_GAS_LIMIT);
```

CVF-9 INFO

- **Category** Suboptimal
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description The "mainOk" flag is checked twice.

Recommendation Refactor like this: if (!mainOk) { if (!refOk) revert ...; else return (refData...); } else if (!refOk) return (mainData...);

Client Comment *We use an early revert / return pattern here for readability.*

```
57 if (!mainOk && !refOk) revert BothAdaptersFailed();
```

```
60 if (mainOk != refOk) {
     return mainOk
```

CVF-10 INFO

- **Category** Readability
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description This code seems overcomplicated.

Recommendation Rewrite as: return value1 == value2 ? 0 : value1 < value2 ? (value2 - value1) * 10000 / value2 : (value1 - value2) * 10000 / value1;

Client Comment *We prefer to avoid the logic duplication.*

```
89 if (value1 == value2) return 0;
90 (uint256 maxVal, uint256 minVal) = value1 > value2 ? (value1, value2
      ↪ ) : (value2, value1);
   return ((maxVal - minVal) * 10_000) / maxVal;
```

CVF-11 FIXED

- **Category** Readability
- **Source** MultiFeedAdapterWithoutRoundsWithReference.sol

Description Outer brackets are redundant.

```
91 return ((maxVal - minVal) * 10_000) / maxVal;
```

ABDK
Consulting

About us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and co-authored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, JavaScript, and other languages.

Contact

Email

dmitry@abdkconsulting.com

Website

abdk.consulting

Twitter

twitter.com/ABDKconsulting

LinkedIn

linkedin.com/company/abdk-consulting